

Distributed Observability Co-Pilot

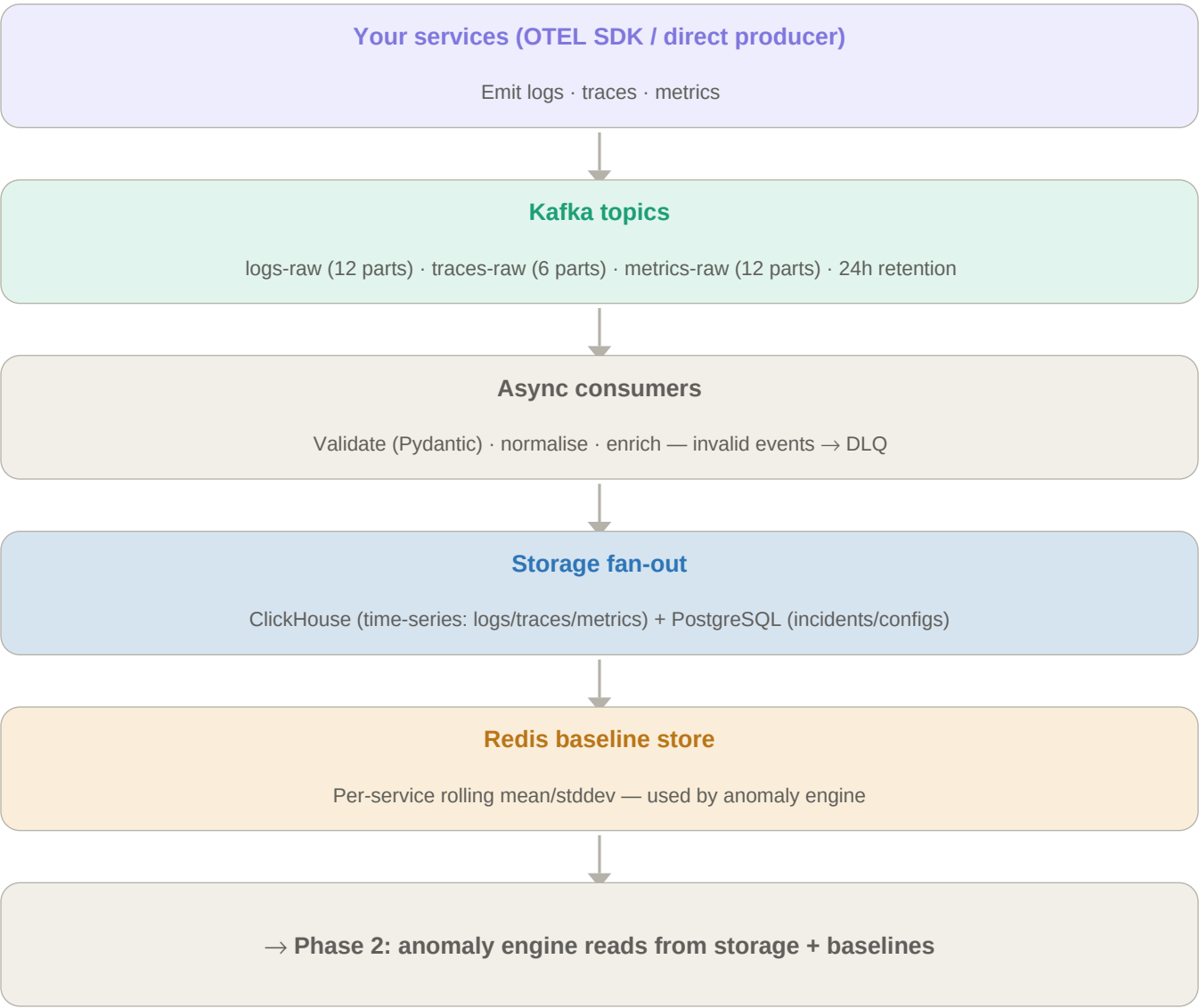
Application Flow

End-to-end walkthrough of the three-phase data pipeline

Phase	Name	What happens
1	Ingestion	Services emit OTEL telemetry → Kafka → consumers → ClickHouse + PostgreSQL
2	Detection & AI	Anomaly engine detects deviations → LLM generates structured RCA
3	API & Dashboard	FastAPI exposes REST + WebSocket → React dashboard + alert consumers

Phase 1 — Telemetry Ingestion

Your services emit logs, traces, and metrics via the OpenTelemetry SDK directly into three dedicated Kafka topics. Python asyncio consumers read each topic, validate every event against a Pydantic schema, normalise the fields, and write to storage. Malformed events go to a Dead Letter Queue instead of being silently dropped. Redis holds per-service rolling baselines that the anomaly engine will query in Phase 2.

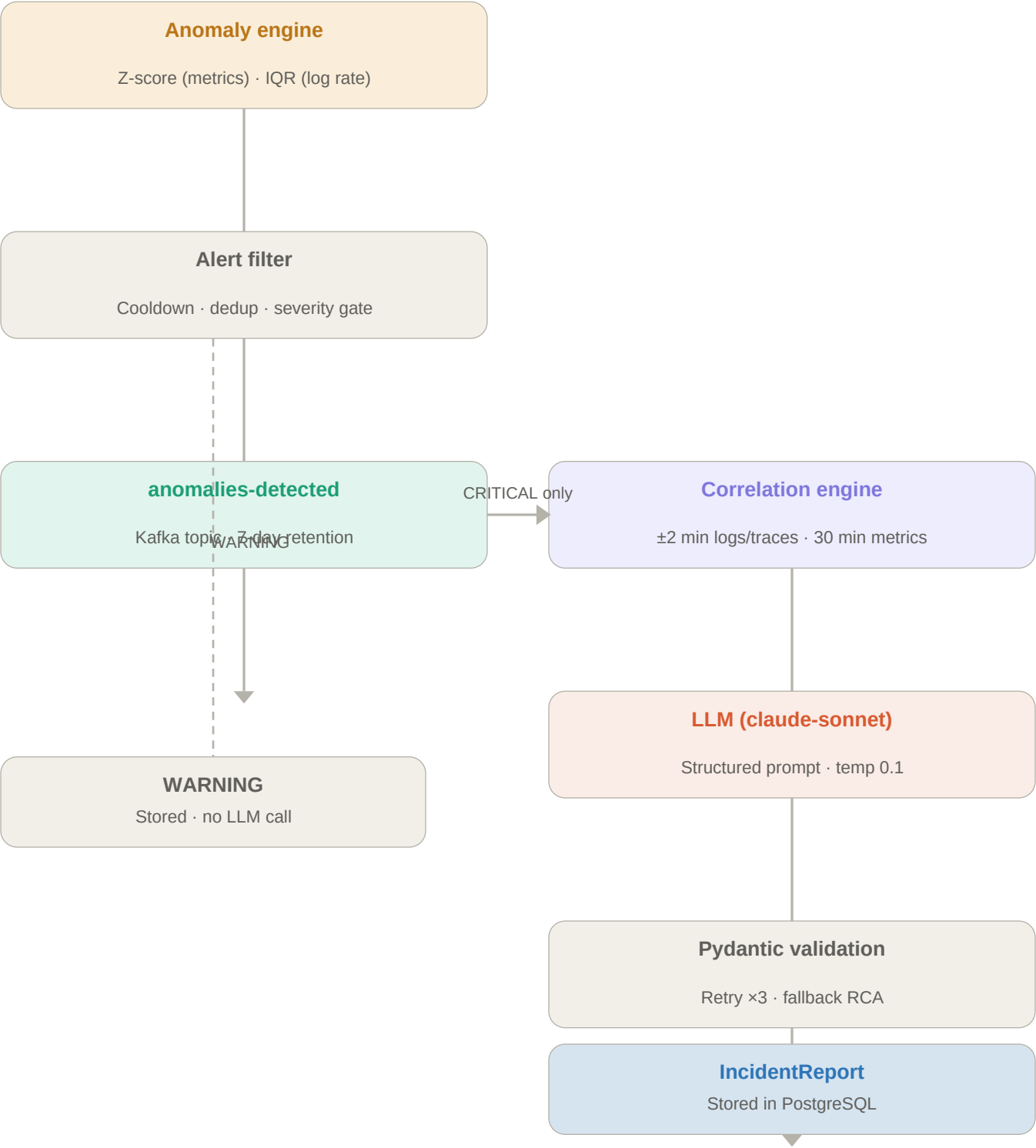


Step	Component	Key detail
1	OTEL producers	JSON or Protobuf · burst up to 50k events/sec · DLQ on validation failure
2	Kafka	acks=all · min.insync.replicas=2 · lz4 compression · auto.create.topics=false
3	Consumers	aiokafka · manual offset commit · batch size 500 · /health endpoint per instance

4	ClickHouse	MergeTree · columnar · TTL 24h · partition by date · ORDER BY (service, timestamp)
5	PostgreSQL	ACID · incidents table · alert_configs table · 90-day retention on incidents

Phase 2 — Anomaly Detection & AI Root Cause Analysis

The anomaly engine continuously compares incoming metrics against each service's rolling baseline stored in Redis, using Z-score for numeric metrics and IQR for log error rates. When a deviation is detected, an alert filter applies three checks — cooldown period, deduplication, and severity gating. WARNING anomalies are stored directly; only CRITICAL ones are published to the *anomalies-detected* Kafka topic. A separate async worker pool (decoupled from ingestion) consumes this topic, assembles correlated context (200 log lines $\pm$ 2 min, all trace spans $\pm$ 2 min, 30 min of metrics), and sends a structured prompt to Claude. The LLM response is validated by Pydantic, retried up to 3 times on failure, and stored as a structured IncidentReport.

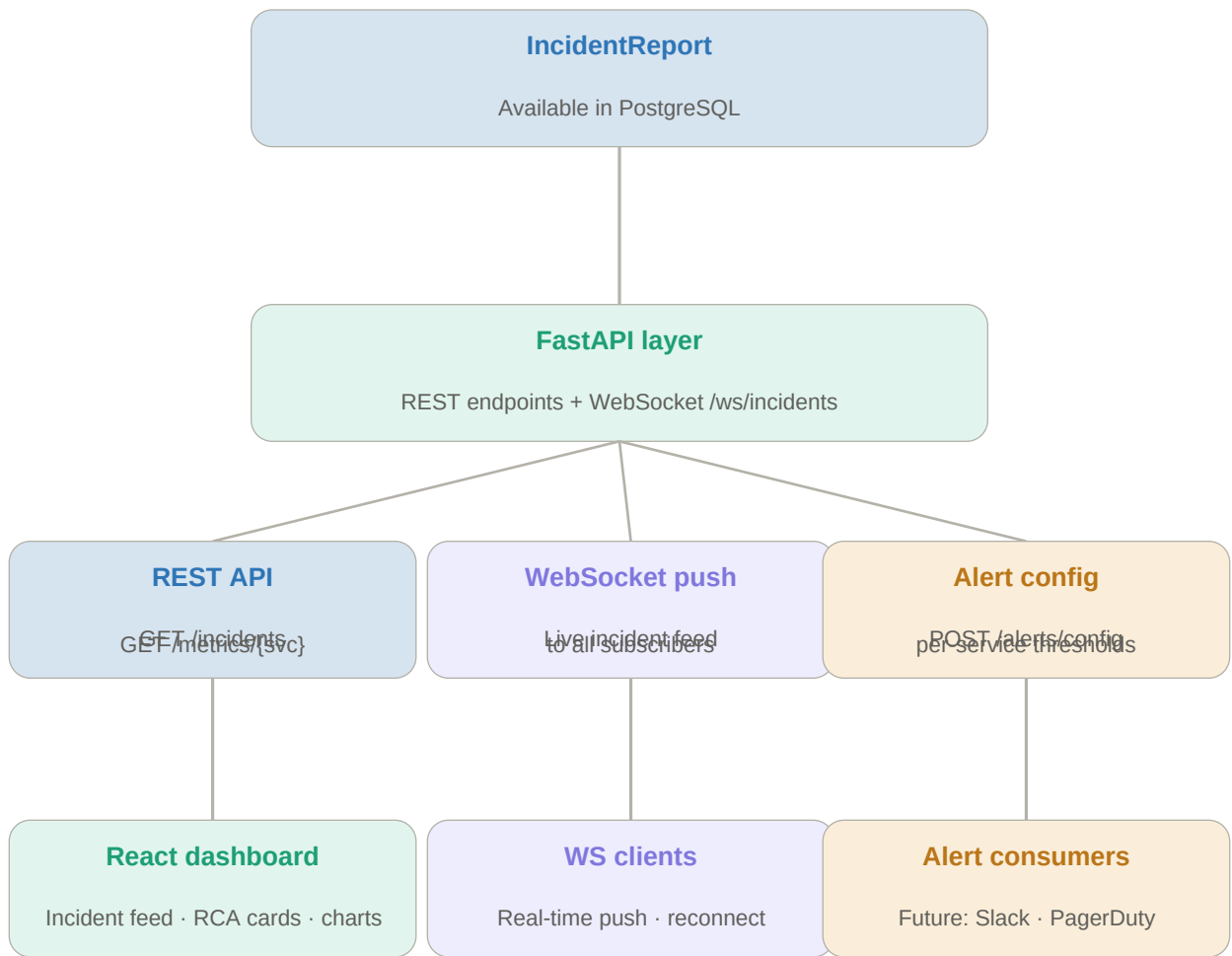


Component	Algorithm / Config	Key detail
Anomaly engine	Z-score (metrics)	Threshold: $ z \geq 3.0 \rightarrow \text{CRITICAL}$, $\geq 2.0 \rightarrow \text{WARNING}$ · min 30s duration
Anomaly engine	IQR (log error rate)	Upper fence = $Q3 + 1.5 \times \text{IQR}$ per 1-min bucket · rolling 60-min window

Alert filter	Cooldown	No re-alert for same service + type within 5 minutes (configurable)
Alert filter	Severity gate	Only CRITICAL triggers LLM call — WARNING stored to DB only
Correlation engine	Context assembly	±2 min logs (200 lines) · ±2 min trace spans · 30 min metrics
LLM layer	claude-sonnet-4-6	temp=0.1 · max_tokens=2048 · timeout=30s · retry×3 · fallback RCA
Pydantic parser	Response schema	root_cause · contributing_factors · remediation_steps · confidence_score

Phase 3 — API & Dashboard Delivery

Once an IncidentReport is written to PostgreSQL, the FastAPI layer makes it immediately available via both REST endpoints and a live WebSocket push. The React dashboard subscribes to the WebSocket and updates in real time — showing the incident feed, expandable RCA cards, time-series charts, and a thumbs up/down rating control. Engineers can also query historical incidents via the REST API or configure per-service alert thresholds without redeploying anything.



Endpoint / Component	Method	Description
GET /api/incidents	REST	Paginated list · filter by service, severity, time range
GET /api/incidents/{id}	REST	Full incident with RCA, contributing factors, remediation steps
GET /api/metrics/{service}	REST	Time-series query with avg/max/min/p95 aggregation
GET /api/health	REST (public)	Health check: Kafka · ClickHouse · PostgreSQL · LLM connectivity

POST /api/alerts/config	REST	Create / update per-service alert thresholds
WS /ws/incidents	WebSocket	Real-time push: incident.created · incident.resolved events
React dashboard	Frontend	Incident feed · RCA cards · metric charts · ratings · health overview